AFF Phase 3 — Global Classes Management

Implementation Plan

Date: 2026-06-14 **Target version:** 2.0.0 **Elementor baseline:** 4.1.3 (storage reworked in 4.1.0 — CPT model)
Status: Planning

Table of Contents

---

1. Elementor V4 Classes — Technical Architecture

1.1 Storage Model (Elementor 4.1.0+)

Each global class is **its own WordPress post** of type `e_global_class` (not kit meta). The kit post holds only the index.

> **Warning:** Pre-4.1.0 sites stored classes in kit meta `_elementor_global_classes` as a single JSON blob. AFF must detect the DB version and handle both. DB version is stored in WP option `elementor_global_classes_db_version` (current = 3).

CPT post meta per class:

Meta key	Content
`_elementor_global_class_id`	The `g-XXXXXXX` string ID
`_elementor_global_class_data`	`{type, variants, sync_to_v3?}` — published/frontend state

Meta key	Content
`_elementor_global_class_data_preview`	Same structure — draft/editor state
`_elementor_version`	Elementor version at last save
`_elementor_global_class_edited`	Unix timestamp of last edit

Kit post meta (index layer on the active Elementor kit):

Meta key	Content
`_elementor_global_classes_order`	`{order: ['g-xxx', 'g-yyy', …]}`
`_elementor_global_classes_labels`	`{g-xxx: 'Label', …}`
`_elementor_global_classes_post_ids`	`{g-xxx: 1234, …}` (class ID → WP post ID)
`_elementor_global_classes_sync_to_v3`	`{g-xxx: true, …}`
`…_preview` variants	Same four keys suffixed `_preview` for editor draft state

1.2 Class Object Structure

```
{
  "id": "g-8091449",
  "label": "Primary Button",
  "type": "class",
  "variants": [
    {
      "meta": {
        "breakpoint": "desktop",
        "state": null
      },
      "props": { /* typed prop objects — see §2 */ },
      "custom_css": { "raw": "<escaped CSS string>" }
    },
    {
      "meta": { "breakpoint": "tablet", "state": null },
      "props": { … },
      "custom_css": null
    },
    {
      "meta": { "breakpoint": "desktop", "state": "hover" },
      "props": { … },
      "custom_css": null
    }
  ],
  "sync_to_v3": false
}
```

1.3 Class IDs

- Format: `g-` prefix + unique numeric/random suffix (e.g. `g-8091449`)
- **Client-generated** — the server stores whatever ID it receives
- Must be unique across all classes; server does not mint IDs
- Used as the CSS class selector: `.elementor .g-8091449 { … }`
- Label naming rules: 2–50 chars, `[a-zA-Z0-9_-]` only, no spaces, cannot start with digit or `--`, `container` is reserved

1.4 Breakpoints

Elementor breakpoint slugs (from `core/breakpoints/manager.php`):

Slug	Notes
`desktop`	Base breakpoint — no media query wrapper emitted
`tablet`	Always enabled
`mobile`	Always enabled
`laptop`	Optional — user must enable in Elementor settings
`tablet_extra`	Optional
`mobile_extra`	Optional
`widescreen`	Optional

> Desktop is `"desktop"` (a string), **not** `null`. The `state` field uses `null` for normal. Breakpoint is not yet server-validated against the registered set (Elementor TODO EDS-528) — a bogus slug will be stored but will never render.

1.5 States

Valid values for `meta.state` (from `style-states.php`):

Value	Renders as	Notes
`null`	(no suffix)	Normal/default
`"hover"`	`:hover`	Also auto-pairs with `focus-visible`
`"active"`	`:active`	
`"focus"`	`:focus`	
`"focus-visible"`	`:focus-visible`	
`"checked"`	`:checked`	
`"e--selected"`	`.e--selected`	Class, not pseudo — renders as CSS class selector

1.6 `sync_to_v3` Flag

When `true`, the class participates in Elementor's V3↔V4 design system bridge. Only affects published/frontend context. Most classes do not need this. AFF exposes it as a simple toggle per class.

2. Critical Discovery: Props Are Typed Objects, Not CSS Strings

This is the single most important architectural finding.

The `props` object in each variant does **not** contain raw CSS strings. Each property value is a typed object from Elementor's atomic prop-type system:

```
"props": {
  "font-size": { "$$type": "size", "value": { "size": 16, "unit": "px" } },
  "color":     { "$$type": "color", "value": { "color": "#ffffff" } },
  "padding":   { "$$type": "linked-dimensions", "value": { "top": "12px", "right":
"24px", "bottom": "12px", "left": "24px" } }
}
```

The props schema is validated server-side against `style-schema.php` via `Props_Parser`. Unknown or malformed props are **dropped silently** or rejected.

However: each variant also has an optional `custom_css` field:

```
"custom_css": { "raw": "<escaped arbitrary CSS declaration block>" }
```

This accepts arbitrary CSS declarations (sanitized with `sanitize_textarea_field`, encoded with `Utils::encode_string`). It is **not** validated against the schema.

3. AFF Strategy Decision: Two Editing Modes

The typed props system requires deep knowledge of every prop-type schema to build a full structured editor. That is a large, fragile surface area tied to Elementor internals.

Recommended approach — two modes:

Mode A: Raw CSS Editor (Phase 3)

Edit via `custom_css.raw` only. User writes CSS declarations in a textarea-like editor. AFF stores the raw CSS, encodes it on commit, decodes it on sync.

- Pros: Works for any CSS property, no schema dependency, implementable in Phase 3
- Cons: No type-aware controls (no color pickers, no unit selectors for this mode)
- AFF never reads or writes `props` — it preserves whatever `props` arrived from Elementor but does not allow AFF to modify them. If a class has existing `props`, they are shown as read-only, and the user edits via `custom_css.raw`.

Mode B: Structured Props Editor (Phase 4 or later)

Type-aware editing for each prop — color picker for `$$type: color`, size+unit for `$$type: size`, etc. Requires mapping every prop-type and building corresponding UI controls.

> **Phase 3 ships Mode A only.** This is the pragmatic choice. Mode B is a separate, large feature — plan it after Mode A ships and actual class-editing usage is understood.

---

4. AFF Data Model

4.1 How AFF Stores Class Data

AFF stores a project `.aff.json` snapshot that extends the existing format:

```
{
  "project": "my-brand",
  "variables": { … },
  "classes": {
    "items": {
      "g-8091449": {
        "id": "g-8091449",
        "label": "Primary Button",
        "type": "class",
        "variants": [ … ],
        "sync_to_v3": false,
        "_aff": {
          "category": "Buttons",
          "tags": ["interactive", "brand"],
          "notes": "Main CTA button style",
          "status": "synced",
          "syncedHash": "abc123"
        }
      }
    },
    "order": ["g-8091449", "g-1234567"],
    "categories": [
      { "id": "cat-1", "name": "Buttons", "order": 0 },
      { "id": "cat-2", "name": "Typography", "order": 1 }
    ]
  }
}
```

All `_aff` fields are AFF-only metadata — they are stripped before committing to Elementor. The `status` and `syncedHash` fields track sync state, same pattern as variables.

4.2 Status Model

Status	Meaning
`synced`	AFF matches what Elementor has
`modified`	AFF differs from last synced state
`new`	Created in AFF, not yet committed

Status	Meaning
`orphaned`	In AFF but deleted from Elementor since last sync

Hash is computed from the serialized variants (excluding `_aff`). Status dots appear in the class list row, same as variable rows.

---

5. Left Column Changes

The existing Classes nav item becomes a real expandable tree:

```
▾ Classes  (45)
    All  (45)
  ▾ Buttons  (8)
  ▾ Typography  (12)
  ▾ Layout  (10)
  ▾ Visual Effects  (7)
    Uncategorized  (8)
```

- Category nodes are user-defined (same CRUD as variable categories)
- Count in parentheses = number of classes in that category
- "All" node at top = flat list of every class regardless of category
- Clicking any node opens that class list in the edit space
- Tag filtering is in the edit-space filter bar, not the left column

Left column tree changes in `aff-panel-left.js`:

- Activate the Classes section with real data
- Render category sub-nodes under Classes
- Wire click → `AFF.EditSpace.openClasses(categoryId)`

---

6. Edit Space — Class List View

The class list view is similar to the variable category block but with different columns.

6.1 Class Row Layout

```
[drag] [status] [label/name] [variant count] [breakpoints] [usage] [expand]
[delete]
```

Col	Content
Drag handle	Drag to reorder within category
Status dot	synced / modified / new / orphaned (with tooltip)

Col	Content
Label	Editable inline; `.class-name` shown in muted text below
Variants	Count of variants defined (e.g. "3 variants")
Breakpoints	Chips showing which breakpoints have definitions (D T M icons)
Usage	Number of widgets using this class
Expand	Opens the class expand panel
Delete	Trash icon on hover, confirmation modal

Grid: `24px 8px 1fr 80px 80px 48px 28px 28px` (8 columns, consistent with variables).

6.2 Filter Bar

Same as variables: search, sort, status filter. Add:

- **Tag filter** — dropdown/chips to filter by tag
- **Breakpoint filter** — show only classes that have a definition for a given breakpoint

6.3 Status Legend

Same status legend row as variables (Synced / Modified / New / Orphaned).

6.4 Category Header

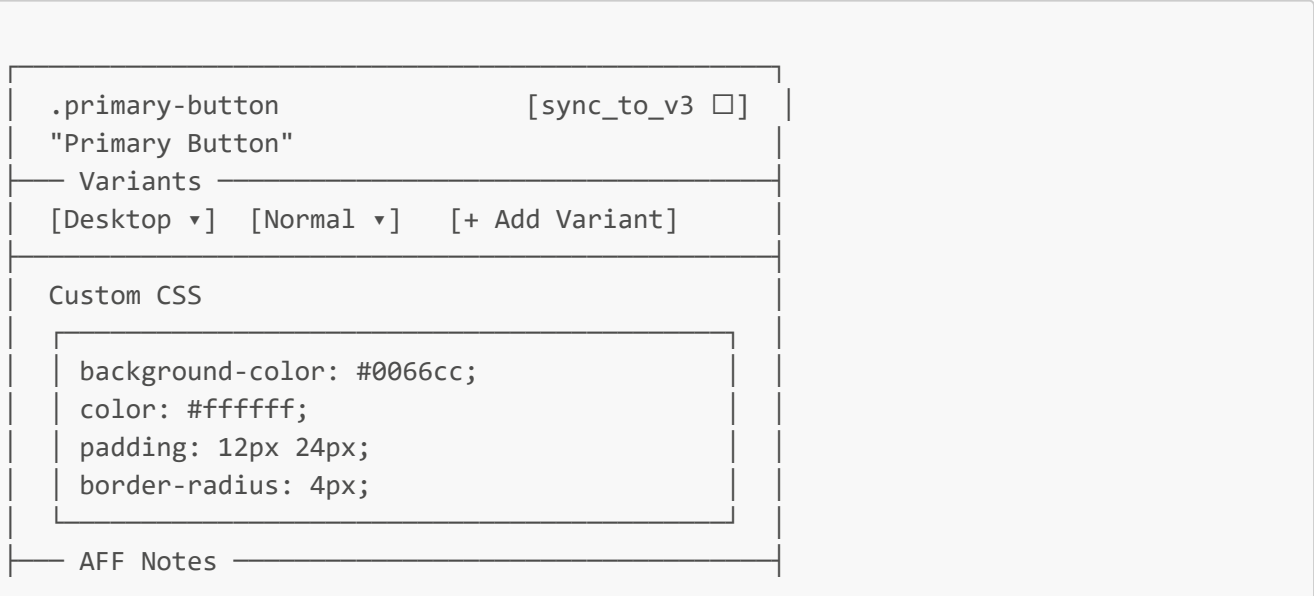
Same as variable category header (collapse, name, count, add-class button). Sub-categories: **not in Phase 3** for classes. Single-level categories only.

---

7. Edit Space — Class Expand Panel

The expand panel is where classes are actually edited.

7.1 Panel Structure



```
│  [optional developer notes field]                │
├── Category / Tags ──────────────────────────────┤
│  Category: [Buttons ▾]                          │
│  Tags: [interactive ×] [brand ×] [+ add tag]    │
└──────────────────────────────────────────────────┘
```

7.2 Variant Selector

Two dropdowns: **Breakpoint** and **State**. Changing either switches the active variant. The CSS editor below shows the `custom_css.raw` for the selected variant.

If a variant exists for the selected combination, its CSS appears. If not, the editor is empty — saving non-empty CSS creates the variant. Clearing a variant's CSS (save empty) removes that variant.

Breakpoints shown: only enabled breakpoints from the Elementor configuration. AFF reads the enabled breakpoint set from Elementor's breakpoints config on sync.

7.3 Existing `props` Handling

If Elementor has `props` data in a variant (from editing in the Elementor editor):

- Display a read-only summary: "Elementor-structured props present (read only)"
- List the prop keys as a chip row so the user can see what's there
- AFF preserves these `props` as-is on commit — never overwrites or strips them
- User can add `custom_css` on top; both `props` and `custom_css` are sent together

This prevents AFF from destroying structured props written by the Elementor editor.

7.4 CSS Editor Component

A plain `<textarea>` with monospace font, line numbers (CSS-specific line numbering is bonus — plain textarea is acceptable for Phase 3), auto-resize.

Validation on save: strip the selector wrapper if the user accidentally included it (e.g. if they pasted `.myclass { … }`, strip to just the declarations). Warn in UI.

Encoding: AFF stores decoded CSS in memory and in `.aff.json`. Encoding for Elementor (`Utils::encode_string`) is applied only on commit.

---

8. Sync — Read from Elementor

8.1 Read Path (PHP, server-side)

Do **not** call the REST endpoint over HTTP. Use the repository directly to avoid nonce issues:

```php
use Elementor\Modules\GlobalClasses\Global_Classes_Repository;

$kit    = \Elementor\Plugin::$instance->kits_manager->get_active_kit();
$repo   = Global_Classes_Repository::make( $kit );
```

```
$labels = $repo->all_labels();              // lightweight: id => label
$items  = $repo->get_by_ids( array_keys( $labels ) ); // full data for all classes
$order  = /* read _elementor_global_classes_order kit meta */;
```

> **Never call `$repo->all()`** — explicitly flagged in Elementor source as "too heavy — may cause server to freeze." Always use `all_labels()` + `get_by_ids()`.

8.2 Sync AJAX Endpoint

New AJAX action: `aff_sync_from_elementor_classes`

Returns:

```
{
  "items": { "g-xxx": { …class object… } },
  "order": ["g-xxx", "g-yyy"],
  "enabledBreakpoints": ["desktop", "tablet", "mobile"]
}
```

8.3 Sync Options

Reuse the existing sync options modal:

- **Sync by name** — match incoming classes to AFF classes by label; preserve category/tag assignments
- **Clear and replace** — replace all AFF class data with Elementor's current state

8.4 Status After Sync

Compute hash of incoming variant data; compare to stored `syncedHash`. Set status: `synced` if matching, `modified` if AFF has changes, update `syncedHash`.

---

9. Commit — Write to Elementor

9.1 Write Path (PHP, server-side)

Use `Global_Classes_Repository::put()` directly. This is the safest approach:

- Handles CPT create/update/delete
- Updates all kit meta (order, labels, post-ID map, sync-to-v3 map)
- Fires `elementor/global_classes/update` and cleanup actions
- Clears preview meta to keep editor in sync

```
$repo->put( $items_array, $order_array );
```

9.2 Pre-Commit Processing

Before calling `put()`:

1. Strip all `_aff` keys from each class object
2. For each variant, encode `custom_css.raw` using Elementor's `Utils::encode_string()`
3. Preserve existing `props` exactly as received from last sync — never regenerate or modify
4. Validate class IDs follow `g-` format and are unique

9.3 Commit Summary Dialog

Show before writing:

- Classes being added (new in AFF, not in Elementor)
- Classes being modified (variants changed in AFF)
- Classes being deleted (orphaned, with confirmation)
- Classes unchanged (status: synced)

User must confirm before `put()` is called.

9.4 Capability Check

Before the AJAX handler proceeds, verify the requesting user has:

- `manage_options` (AFF's standard check)
- `elementor_global_classes_update_class` (Elementor's specific cap)

If the second cap is missing, show a clear error: "Your user role does not have permission to update Elementor Global Classes."

---

10. Category System

Single-level categories only (no sub-categories in Phase 3).

Stored in: `.aff.json` under `classes.categories[]` **Not sent to Elementor** — categories are AFF metadata only.

10.1 Category CRUD

Same as variable categories:

- Add category (button at bottom of category list in left nav)
- Rename inline
- Delete with count warning ("This category contains 8 classes. Classes will move to Uncategorized.")
- Drag to reorder categories in left nav (reorders `.aff.json` categories array)

10.2 Auto-Categorization on Sync

When classes are synced from Elementor and are new to AFF, offer auto-categorization. Heuristic (suggest only, user confirms):

- Has `color`, `background-color`, `font-*` props or custom CSS → suggest "Typography"
- Has `padding`, `margin`, `display`, `flex-*` → suggest "Layout"

- Has `border`, `box-shadow`, `opacity` → suggest "Visual"
- Anything else → "Uncategorized"

AFF presents a pre-categorized list in the sync modal. User can adjust before accepting.

---

11. Tag System

Tags are AFF-only metadata — not sent to Elementor.

Stored in: `_aff.tags` array on each class in `.aff.json`

11.1 Tag UI

- Tags displayed as chips in the expand panel
- Click `×` on a chip to remove that tag
- Type in a tag input to add; suggest existing tags from autocomplete
- No tag management screen needed — tags are created by use, removed when no longer used

11.2 Tag Filtering

Filter bar in the class list view includes a tag dropdown. Selecting a tag shows only classes carrying that tag. Multiple tags filter as AND (class must have all selected tags).

Tags do **not** appear in the left column tree (categories only in left nav).

---

12. Export / Import

12.1 Export

Extend `.aff.json` format — existing export already writes the full project file. Classes are included automatically when the `classes` block is populated.

CSS export (new option):

- Export button → modal: "Export Variables CSS / Export Classes CSS / Export Both"
- Classes CSS export emits one CSS rule per class per variant:

```
/* Primary Button */
.g-8091449 { background-color: #0066cc; color: #fff; }
.g-8091449:hover { background-color: #0052a3; }
@media (max-width: 1024px) { .g-8091449 { padding: 10px 20px; } }
```

  Breakpoint → media query mapping is stored in AFF at sync time from Elementor's config.
- CSS export does not import back into Elementor — it is for reference/portability only.

12.2 Import

Import from `.aff.json` — same as variables. If the file contains a `classes` block, those classes are loaded. Merge or replace options (same sync modal pattern).

---

13. Usage Scanning

Elementor already tracks usage via `_elementor_used_global_class` post meta on pages/posts.

AFF AJAX: `aff_get_class_usage_counts`

PHP handler queries:

```
global $wpdb;
$results = $wpdb->get_results(
    "SELECT meta_value AS class_id, COUNT(post_id) AS usage_count
     FROM {$wpdb->postmeta}
     WHERE meta_key = '_elementor_used_global_class'
     GROUP BY meta_value"
);
```

Returns `{ "g-8091449": 14, "g-1234567": 3 }`. Displayed in the class row as a count badge. Same "run scan" button pattern as variables.

---

14. Implementation Milestones

Milestone 3.1 — Foundation and Read-Only Display (no editing)

- PHP: `aff_sync_from_elementor_classes` AJAX endpoint (read via Repository)
- PHP: `aff_get_class_usage_counts` AJAX endpoint
- PHP: DB version detection (`elementor_global_classes_db_version`)
- JS: `aff-classes.js` module — class list render, left nav activation
- JS: Left column — Classes section with category nodes and counts
- JS: Edit space — class list view (rows: status, label, variant count, breakpoints, usage, expand, delete)
- JS: Filter bar — search, status filter, tag filter (no tags yet, just the infrastructure)
- JS: Sync modal extension — add Classes sync option
- CSS: `aff-classes.css` — class row grid, expand panel shell
- Data: `.aff.json` schema extended with `classes` block
- No commit, no editing — display and organize only

Milestone 3.2 — Category and Tag Management

- Category CRUD for classes (same CatMixin pattern as variables)
- Left nav category nodes wired to category filter
- Auto-categorization suggestion on sync
- Tag add/remove in expand panel
- Tag filter in filter bar
- Category/tag data persisted in `.aff.json`

Milestone 3.3 — Class Editing and Commit

- Expand panel — variant selector (breakpoint + state dropdowns)
- CSS editor textarea — edit `custom_css.raw` per variant
- Variant add / remove
- `sync_to_v3` toggle
- Notes field
- Status tracking (modified/synced/new/orphaned)
- PHP: `aff_commit_classes_to_elementor` AJAX endpoint (write via Repository)
- Capability check for `elementor_global_classes_update_class`
- Pre-commit encoding of `custom_css.raw`
- Commit summary dialog
- Undo/Redo integration

Milestone 3.4 — Polish and Integration

- CSS export for classes
- Import from `.aff.json` (classes block merge/replace)
- Print/PDF — extend to include classes view
- Responsive class row (hide breakpoint chips, collapse variant count on narrow viewport)
- Existing `props` read-only display in expand panel

---

15. File Structure Changes

New files:

```
admin/
  css/
    aff-classes.css            # Class list view, expand panel, variant editor
  js/
    aff-classes.js             # Classes module (mirrors aff-colors.js structure)
```

Modified files:

```
admin/js/
  aff-app.js                   # Register Classes module, extend init
  aff-edit-space.js            # Route Classes nav clicks to aff-classes.js
  aff-panel-left.js            # Activate Classes section, render category nodes
  aff-panel-right.js           # Extend sync panel to include Classes sync
  aff-panel-top.js             # Extend print modal to include Classes view option
admin/css/
  aff-colors.css               # Minor: extract any shared expand-panel CSS to
shared file
includes/
  class-aff-admin.php          # Enqueue aff-classes.css + aff-classes.js
  class-aff-ajax-handler.php   # Add: aff_sync_from_elementor_classes,
                               #      aff_commit_classes_to_elementor,
```

```
                                #      aff_get_class_usage_counts
  data/
    aff-defaults.json           # Add default class categories list
```

---

16. Known Risks and Gotchas

Storage version drift

Pre-4.1.0 sites have classes in kit meta; 4.1.0+ use CPT. AFF must check `elementor_global_classes_db_version` before deciding how to read. If < 3, fall back to reading the legacy `_elementor_global_classes` kit meta key.

Preview vs. frontend duality

Every store has a `_preview` twin. The Elementor editor reads/writes preview context. AFF operates in frontend context only. If a user has the Elementor editor open while AFF commits, the editor may show stale preview data. Document this as a known limitation. Workaround: close Elementor editor before committing from AFF.

`$repo->all()` is dangerous

Explicitly flagged in Elementor source. Never call it. Always use `all_labels()` + `get_by_ids()`.

Post-ID map can drift

If a class post is created by bypassing the repository, the post-ID map in kit meta can get out of sync. The repository self-heals by re-querying, but it also prunes duplicate posts (keeps lowest post ID). AFF must always write through the repository, never create `e_global_class` posts directly.

Breakpoints not server-validated

Elementor does not yet validate the breakpoint slug against the registered set (TODO EDS-528). A bad slug is stored silently and never renders. AFF must validate against the enabled breakpoint list it received at sync time before allowing a variant to be saved.

Encode/decode for `custom_css.raw`

Elementor encodes raw CSS using `Utils::encode_string()` before storage and decodes with `Utils::decode_string()` on read. AFF must use the same utility (call via PHP) when committing. The `.aff.json` stores decoded (human-readable) CSS. Encoding is applied only at commit time.

Label naming rules

Labels are validated: `[a-zA-Z0-9_-]` only, 2–50 chars, no spaces, cannot start with digit or `--`. Duplicate labels are auto-renamed by Elementor with `DUP_` prefix. AFF should validate labels client-side before commit and warn on duplicates rather than letting Elementor silently rename.

Editor-count limit

The Elementor editor enforces a class count limit (community reports ~100, server constant is 1000). AFF does not enforce this limit, but the user should be warned if their project contains >100 classes.

Capability: `elementor_global_classes_update_class`

This capability is granted to `administrator` role only by Elementor's migration. Lower-privileged admin users (Editor role with `manage_options`) may lack it. AFF must check for it explicitly and surface a useful error, not a silent failure.

`custom_css` vs `props` coexistence

A class can have both `props` and `custom_css` in the same variant. AFF reads but does not modify `props`; it reads and writes `custom_css`. Both must be preserved on round-trip. Never overwrite the `props` key when committing AFF changes.

Experiment gates

Global Classes require two Elementor experiments active: `e_classes` and atomic widgets. AFF should detect whether these experiments are enabled and show a clear message if they are not, rather than showing an empty class list.

Appendix: Key Elementor Source Files (local install, 4.1.3)

```
modules/global-classes/
  global-classes-rest-api.php          # REST routes — read for endpoint reference
  global-classes-repository.php        # USE THIS for read/write — the safe API
surface
  global-class-post.php                # CPT post wrapper
  global-class-post-type.php           # CPT registration (post type
'e_global_class')
  global-classes-order.php             # Order kit-meta wrapper
  global-classes-labels.php            # Labels kit-meta wrapper
  global-classes-post-ids.php          # Post-ID map wrapper (self-heals, prunes
duplicates)
  global-classes-parser.php            # Validation for incoming class data
  global-classes-sync-map.php (design-system-sync)   # sync_to_v3 handling
  database/
    global-classes-database-updater.php
    migrations/migrate-to-posts.php    # DB v2 — moved kit-meta blob to CPT
    migrations/reconcile-downgraded-posts.php  # DB v3 — handles upgrade/downgrade
drift
    migrations/add-capabilities.php    # Grants elementor_global_classes_* caps

modules/atomic-widgets/
  parsers/style-parser.php             # Validates variants and props against
schema
  styles/style-schema.php             # Canonical prop schema (what goes in
props{})
  styles/style-states.php             # Valid state slugs
  styles/style-variant.php            # Variant object builder
```

```
  styles/styles-renderer.php          # Emits final CSS

core/breakpoints/manager.php          # Breakpoint slugs and enabled-set
resolution
```